JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DateFormat

java.lang.Object
    java.text.Format
        java.text.DateFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

Direct Known Subclasses:

`SimpleDateFormat`

```
public abstract class DateFormat
extends Format
```

`DateFormat` is an abstract class for date/time formatting subclasses which formats and parses dates or time in a language-independent manner. The date/time formatting subclass, such as `SimpleDateFormat`, allows for formatting (i.e., date → text), parsing (text → date), and normalization. The date is represented as a `Date` object or as the milliseconds since January 1, 1970, 00:00:00 GMT.

`DateFormat` provides static factory methods for obtaining default date/time formatters based on the default or a given locale and a number of formatting styles. The formatting styles include `FULL`, `LONG`, `MEDIUM`, and `SHORT`. For any of the factory methods with the parameter *style*, an `IllegalArgumentException` will be thrown if *style* is not equal to any of the defined formatting styles. More detail and examples of using these styles are provided in the method descriptions.

`DateFormat` helps you to format and parse dates for any locale. Your code can be completely independent of the locale conventions for months, days of the week, or even the calendar format: lunar vs. solar.

To format a date for the current Locale, use one of the static factory methods:

```
myString = DateFormat.getDateInstance().format(myDate);
```

If you are formatting multiple dates, it is more efficient to get the format and use it multiple times so that the system doesn't have to fetch the information about the local language and country conventions multiple times.

```
DateFormat df = DateFormat.getDateInstance();
for (Date myDate : dates) {
    output.println(df.format(myDate) + "; ");
}
```

To format a date for a different Locale, specify it in the call to `getDateInstance()`.

```
DateFormat df = DateFormat.getDateInstance(DateFormat.LONG, Locale.FRANCE);
```

If the specified locale contains "ca" (calendar), "rg" (region override), and/or "tz" (timezone) Unicode extensions, the calendar, the country and/or the time zone for formatting are overridden. If both "ca" and "rg" are specified, the calendar from the "ca" extension supersedes the implicit one from the "rg" extension.

You can use a DateFormat to parse also.

```
myDate = df.parse(myString);
```

Use `getDateInstance` to get the normal date format for that country. There are other static factory methods available. Use `getTimeInstance` to get the time format for that country. Use `getDateTimeInstance` to get a date and time format. You can pass in different options to these factory methods to control the length of the result; from `SHORT` to `MEDIUM` to `LONG` to `FULL`. The exact result depends on the locale, but generally:

- `SHORT` is the shortest and mainly numeric, such as `12.13.52` or `3:30pm`
- `MEDIUM` is longer, such as `Jan 12, 1952`
- `LONG` is even longer, such as `January 12, 1952` or `3:30:32pm`
- `FULL` is the longest, such as `Tuesday, April 12, 1952 AD or 3:30:42pm PST`.

For those fields with text, typically abbreviated text form is used with `MEDIUM` option, and full text form is used with `LONG` and `FULL` options.

You can also set the time zone on the format if you wish. If you want even more control over the format or parsing, (or want to give your users more control), you can try casting the `DateFormat` you get from the factory methods to a `SimpleDateFormat`. This will work for the majority of countries; just remember to put it in a `try` block in case you encounter an unusual one.

You can also use forms of the parse and format methods with `ParsePosition` and `FieldPosition` to allow you to

- progressively parse through pieces of a string.
- align any particular field, or find out where it is for selection on the screen.

Synchronization

Date formats are not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

API Note:

Consider using `DateTimeFormatter` as an immutable and thread-safe alternative.

Implementation Requirements:

- The `format(Date, StringBuffer, FieldPosition)` and `parse(String, ParsePosition)` methods may throw `NullPointerException`, if any of their parameter is `null`. The subclass may provide its own implementation and specification about `NullPointerException`.
- The `setCalendar(Calendar)`, `setNumberFormat(NumberFormat)` and `setTimeZone(TimeZone)` methods do not throw `NullPointerException` when their parameter is `null`, but any subsequent operations on the same instance may throw `NullPointerException`.
- The `getCalendar()`, `getNumberFormat()` and `getTimeZone()` methods may return `null`, if the respective values of this instance is set to `null` through the corresponding setter methods. For Example: `getTimeZone()` may return `null`, if the `TimeZone` value of this instance is set as `setTimeZone(null)`.

Since:

1.1

See Also:

`Format`, `NumberFormat`, `SimpleDateFormat`, `Calendar`, `GregorianCalendar`, `TimeZone`, `DateTimeFormatter`, Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
`static class`	**`DateFormat.Field`**	Defines constants that are used as attribute keys in the `AttributedCharacterIterator` returned from `DateFormat.formatToCharacterIterator` and

	as field identifiers in `FieldPosition`.

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	**`AM_PM_FIELD`**	Useful constant for AM_PM field alignment.
`protected Calendar`	**`calendar`**	The `Calendar` instance used for calculating the date-time fields and the instant of time.
`static final int`	**`DATE_FIELD`**	Useful constant for DATE field alignment.
`static final int`	**`DAY_OF_WEEK_FIELD`**	Useful constant for DAY_OF_WEEK field alignment.
`static final int`	**`DAY_OF_WEEK_IN_MONTH_FIE`**	Useful constant for DAY_OF_WEEK_IN_MONTH field alignment.
`static final int`	**`DAY_OF_YEAR_FIELD`**	Useful constant for DAY_OF_YEAR field alignment.
`static final int`	**`DEFAULT`**	Constant for default style pattern.
`static final int`	**`ERA_FIELD`**	Useful constant for ERA field alignment.
`static final int`	**`FULL`**	

		Constant for full style pattern.
static final int	**HOUR_OF_DAY0_FIELD**	Useful constant for zero-based HOUR_OF_DAY field alignment.
static final int	**HOUR_OF_DAY1_FIELD**	Useful constant for one-based HOUR_OF_DAY field alignment.
static final int	**HOUR0_FIELD**	Useful constant for zero-based HOUR field alignment.
static final int	**HOUR1_FIELD**	Useful constant for one-based HOUR field alignment.
static final int	**LONG**	Constant for long style pattern.
static final int	**MEDIUM**	Constant for medium style pattern.
static final int	**MILLISECOND_FIELD**	Useful constant for MILLISECOND field alignment.
static final int	**MINUTE_FIELD**	Useful constant for MINUTE field alignment.
static final int	**MONTH_FIELD**	Useful constant for MONTH field alignment.
protected NumberFormat	**numberFormat**	The number formatter that `DateFormat` uses to format numbers in dates and times.
static final int	**SECOND_FIELD**	Useful constant for SECOND field alignment.
static final int	**SHORT**	Constant for short style pattern.

static final int	**TIMEZONE_FIELD**	Useful constant for TIMEZONE field alignment.
static final int	**WEEK_OF_MONTH_FIELD**	Useful constant for WEEK_OF_MONTH field alignment.
static final int	**WEEK_OF_YEAR_FIELD**	Useful constant for WEEK_OF_YEAR field alignment.
static final int	**YEAR_FIELD**	Useful constant for YEAR field alignment.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**DateFormat**()	Create a new date format.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
Object	**clone**()	Overrides Cloneable
boolean	**equals**(Object obj)	Compares the specified object with this DateFormat for equality.
final StringBuffer	**format**(Object obj, StringBuffer toAppendTo, FieldPosition fieldPosit	Formats the given Object into a date-time string.
final String	**format**(Date date)	Formats a Date into a date-time string.
abstract StringBuffer	**format**(Date date, StringBuffer toAppendTo, FieldPosition fieldPosit	Formats a Date into a date-time string.

static Locale[]	**getAvailableLocales**()	Returns an array of all locales for which the get*Instance methods of this class can return localized instances.
Calendar	**getCalendar**()	Gets the calendar associated with this date/time formatter.
static final DateFormat	**getDateInstance**()	Gets the date formatter with the default formatting style for the default FORMAT locale.
static final DateFormat	**getDateInstance**(int style)	Gets the date formatter with the given formatting style for the default FORMAT locale.
static final DateFormat	**getDateInstance**(int style, Locale aLocale)	Gets the date formatter with the given formatting style for the given locale.
static final DateFormat	**getDateTimeInstance**()	Gets the date/time formatter with the default formatting style for the default FORMAT locale.
static final DateFormat	**getDateTimeInstance**(int dateStyle, int timeStyle)	Gets the date/time formatter with the given date and time formatting styles for the default FORMAT locale.
static final DateFormat	**getDateTimeInstance**(int dateStyle, int timeStyle, Locale aLocale)	Gets the date/time formatter with the given formatting styles for the given locale.
static final DateFormat	**getInstance**()	Get a default date/time formatter that uses the SHORT style for both the date and the time.

NumberFormat	**getNumberFormat**()	Gets the number formatter which this date/time formatter uses to format and parse a time.
static final DateFormat	**getTimeInstance**()	Gets the time formatter with the default formatting style for the default FORMAT locale.
static final DateFormat	**getTimeInstance** (int style)	Gets the time formatter with the given formatting style for the default FORMAT locale.
static final DateFormat	**getTimeInstance** (int style, Locale aLocale)	Gets the time formatter with the given formatting style for the given locale.
TimeZone	**getTimeZone**()	Gets the time zone.
int	**hashCode**()	Returns the hash code for this DateFormat.
boolean	**isLenient**()	Tell whether date/time parsing is to be lenient.
Date	**parse**(String source)	Parses text from the beginning of the given string to produce a date.
abstract Date	**parse**(String source, ParsePosition pos)	Parse a date/time string according to the given parse position.
Object	**parseObject** (String source, ParsePosition pos)	Parses text from a string to produce a Date.
void	**setCalendar** (Calendar newCalendar)	Set the calendar to be used by this date format.
void	**setLenient** (boolean lenient)	Specify whether or not date/time parsing is to be lenient.

void	**setNumberFormat** (NumberFormat newNumberF	Allows you to set the number formatter.
void	**setTimeZone** (TimeZone zone)	Sets the time zone for the calendar of this DateFormat object.

Methods declared in class Format

format, formatToCharacterIterator, parseObject

Methods declared in class Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Field Details

calendar

protected Calendar calendar

The Calendar instance used for calculating the date-time fields and the instant of time. This field is used for both formatting and parsing.

Subclasses should initialize this field to a Calendar appropriate for the Locale associated with this DateFormat.

numberFormat

protected NumberFormat numberFormat

The number formatter that DateFormat uses to format numbers in dates and times. Subclasses should initialize this to a number format appropriate for the locale associated with this DateFormat.

ERA_FIELD

public static final int ERA_FIELD

Useful constant for ERA field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

YEAR_FIELD

```
public static final int YEAR_FIELD
```

Useful constant for YEAR field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

MONTH_FIELD

```
public static final int MONTH_FIELD
```

Useful constant for MONTH field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

DATE_FIELD

```
public static final int DATE_FIELD
```

Useful constant for DATE field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

HOUR_OF_DAY1_FIELD

```
public static final int HOUR_OF_DAY1_FIELD
```

Useful constant for one-based HOUR_OF_DAY field alignment. Used in FieldPosition of date/time formatting. HOUR_OF_DAY1_FIELD is used for the one-based 24-hour clock. For example, 23:59 + 01:00 results in 24:59.

See Also:

Constant Field Values

HOUR_OF_DAY0_FIELD

```
public static final int HOUR_OF_DAY0_FIELD
```

Useful constant for zero-based HOUR_OF_DAY field alignment. Used in FieldPosition of date/time formatting. HOUR_OF_DAY0_FIELD is used for the zero-based 24-hour clock. For example, 23:59 + 01:00 results in 00:59.

See Also:

Constant Field Values

MINUTE_FIELD

`public static final int MINUTE_FIELD`

Useful constant for MINUTE field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

SECOND_FIELD

`public static final int SECOND_FIELD`

Useful constant for SECOND field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

MILLISECOND_FIELD

`public static final int MILLISECOND_FIELD`

Useful constant for MILLISECOND field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

DAY_OF_WEEK_FIELD

`public static final int DAY_OF_WEEK_FIELD`

Useful constant for DAY_OF_WEEK field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

DAY_OF_YEAR_FIELD

`public static final int DAY_OF_YEAR_FIELD`

Useful constant for DAY_OF_YEAR field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

DAY_OF_WEEK_IN_MONTH_FIELD

```
public static final int DAY_OF_WEEK_IN_MONTH_FIELD
```

Useful constant for DAY_OF_WEEK_IN_MONTH field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

WEEK_OF_YEAR_FIELD

```
public static final int WEEK_OF_YEAR_FIELD
```

Useful constant for WEEK_OF_YEAR field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

WEEK_OF_MONTH_FIELD

```
public static final int WEEK_OF_MONTH_FIELD
```

Useful constant for WEEK_OF_MONTH field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

AM_PM_FIELD

```
public static final int AM_PM_FIELD
```

Useful constant for AM_PM field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

HOUR1_FIELD

```
public static final int HOUR1_FIELD
```

Useful constant for one-based HOUR field alignment. Used in FieldPosition of date/time formatting. HOUR1_FIELD is used for the one-based 12-hour clock. For example, 11:30 PM + 1 hour results in 12:30 AM.

See Also:

Constant Field Values

HOUR0_FIELD

```
public static final int HOUR0_FIELD
```

Useful constant for zero-based HOUR field alignment. Used in FieldPosition of date/time formatting. HOUR0_FIELD is used for the zero-based 12-hour clock. For example, 11:30 PM + 1 hour results in 00:30 AM.

See Also:

Constant Field Values

TIMEZONE_FIELD

```
public static final int TIMEZONE_FIELD
```

Useful constant for TIMEZONE field alignment. Used in FieldPosition of date/time formatting.

See Also:

Constant Field Values

FULL

```
public static final int FULL
```

Constant for full style pattern.

See Also:

Constant Field Values

LONG

```
public static final int LONG
```

Constant for long style pattern.

See Also:

Constant Field Values

MEDIUM

```
public static final int MEDIUM
```

Constant for medium style pattern.

See Also:

Constant Field Values

SHORT

```
public static final int SHORT
```

Constant for short style pattern.

See Also:

Constant Field Values

DEFAULT

```
public static final int DEFAULT
```

Constant for default style pattern. Its value is MEDIUM.

See Also:

Constant Field Values

Constructor Details

DateFormat

```
protected DateFormat()
```

Create a new date format.

Method Details

format

```
public final StringBuffer format(Object obj,
                                 StringBuffer toAppendTo,
                                 FieldPosition fieldPosition)
```

Formats the given `Object` into a date-time string. The formatted string is appended to the given `StringBuffer`.

Specified by:

`format` in class `Format`

Parameters:

`obj` - Must be a `Date` or a `Number` representing a millisecond offset from the Epoch.

`toAppendTo` - The string buffer for the returning date-time string.

`fieldPosition` - keeps track on the position of the field within the returned string. For example, given a date-time text `"1996.07.10 AD at 15:08:56 PDT"`, if the given `fieldPosition` is `YEAR_FIELD`, the begin index and end

index of `fieldPosition` will be set to 0 and 4, respectively. Notice that if the same date-time field appears more than once in a pattern, the `fieldPosition` will be set for the first occurrence of that date-time field. For instance, formatting a `Date` to the date-time string `"1 PM PDT (Pacific Daylight Time)"` using the pattern `"h a z (zzzz)"` and the alignment field `TIMEZONE_FIELD`, the begin index and end index of `fieldPosition` will be set to 5 and 8, respectively, for the first occurrence of the timezone pattern character `'z'`.

Returns:

the string buffer passed in as `toAppendTo`, with formatted text appended.

Throws:

`IllegalArgumentException` - if the `Format` cannot format the given `obj`.

See Also:

`Format`

format

```
public abstract StringBuffer format(Date date,
                                    StringBuffer toAppendTo,
                                    FieldPosition fieldPosition)
```

Formats a `Date` into a date-time string. The formatted string is appended to the given `StringBuffer`.

Parameters:

`date` - a Date to be formatted into a date-time string.

`toAppendTo` - the string buffer for the returning date-time string.

`fieldPosition` - keeps track on the position of the field within the returned string. For example, given a date-time text `"1996.07.10 AD at 15:08:56 PDT"`, if the given `fieldPosition` is `YEAR_FIELD`, the begin index and end index of `fieldPosition` will be set to 0 and 4, respectively. Notice that if the same date-time field appears more than once in a pattern, the `fieldPosition` will be set for the first occurrence of that date-time field. For instance, formatting a `Date` to the date-time string `"1 PM PDT (Pacific Daylight Time)"` using the pattern `"h a z (zzzz)"` and the alignment field `TIMEZONE_FIELD`, the begin index and end index of `fieldPosition` will be set to 5 and 8, respectively, for the first occurrence of the timezone pattern character `'z'`.

Returns:

the string buffer passed in as `toAppendTo`, with formatted text appended.

format

```
public final String format(Date date)
```

Formats a `Date` into a date-time string.

Parameters:

`date` - the time value to be formatted into a date-time string.

Returns:

the formatted date-time string.

parse

```
public Date parse(String source)
           throws ParseException
```

Parses text from the beginning of the given string to produce a date. The method may not use the entire text of the given string.

See the `parse(String, ParsePosition)` method for more information on date parsing.

Parameters:

`source` - A `String` whose beginning should be parsed.

Returns:

A `Date` parsed from the string.

Throws:

`ParseException` - if the beginning of the specified string cannot be parsed.

parse

```
public abstract Date parse(String source,
                           ParsePosition pos)
```

Parse a date/time string according to the given parse position. For example, if `this` has the pattern `"M/d/yy h:m a, z"`, then a time text `"07/10/96 4:5 PM, PDT"` will be parsed into a `Date` that is equivalent to `Date(837039900000L)`.

By default, parsing is lenient: If the input is not in the form used by this object's format method but can still be parsed as a date, then the parse succeeds. Clients may insist on strict adherence to the format by calling `setLenient(false)`.

This parsing operation uses the `calendar` to produce a `Date`. As a result, the `calendar`'s date-time fields and the `TimeZone` value may have been overwritten, depending on subclass implementations. Any `TimeZone` value that

has previously been set by a call to `setTimeZone` may need to be restored for further operations.

Parameters:

`source` - The date/time string to be parsed

`pos` - On input, the position at which to start parsing; on output, the position at which parsing terminated, or the start position if the parse failed.

Returns:

A `Date`, or `null` if the input could not be parsed

parseObject

```
public Object parseObject(String source,
                          ParsePosition pos)
```

Parses text from a string to produce a `Date`.

The method attempts to parse text starting at the index given by `pos`. If parsing succeeds, then the index of `pos` is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed date is returned. The updated `pos` can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of `pos` is not changed, the error index of `pos` is set to the index of the character where the error occurred, and null is returned.

See the `parse(String, ParsePosition)` method for more information on date parsing.

Specified by:

`parseObject` in class `Format`

Parameters:

`source` - A `String`, part of which should be parsed.

`pos` - A `ParsePosition` object with index and error index information as described above.

Returns:

A `Date` parsed from the string. In case of error, returns null.

Throws:

`NullPointerException` - if `source` or `pos` is null.

getTimeInstance

```
public static final DateFormat getTimeInstance()
```

Gets the time formatter with the default formatting style for the default `FORMAT` locale.

This is equivalent to calling `getTimeInstance(DEFAULT, Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a time formatter.

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getTimeInstance

```
public static final DateFormat getTimeInstance(int style)
```

Gets the time formatter with the given formatting style for the default `FORMAT` locale.

This is equivalent to calling `getTimeInstance(style, Locale.getDefault(Locale.Category.FORMAT))`.

Parameters:

`style` - the given formatting style. For example, SHORT for "h:mm a" in the US locale.

Returns:

a time formatter.

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getTimeInstance

```
public static final DateFormat getTimeInstance(int style,
                                               Locale aLocale)
```

Gets the time formatter with the given formatting style for the given locale.

Parameters:

`style` - the given formatting style. For example, SHORT for "h:mm a" in the US locale.

`aLocale` - the given locale.

Returns:

a time formatter.

getDateInstance

```
public static final DateFormat getDateInstance()
```

Gets the date formatter with the default formatting style for the default `FORMAT` locale.

This is equivalent to calling `getDateInstance(DEFAULT, Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a date formatter.

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getDateInstance

```
public static final DateFormat getDateInstance(int style)
```

Gets the date formatter with the given formatting style for the default `FORMAT` locale.

This is equivalent to calling `getDateInstance(style, Locale.getDefault(Locale.Category.FORMAT))`.

Parameters:

`style` - the given formatting style. For example, SHORT for "M/d/yy" in the US locale.

Returns:

a date formatter.

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getDateInstance

```
public static final DateFormat getDateInstance(int style,
                                               Locale aLocale)
```

Gets the date formatter with the given formatting style for the given locale.

Parameters:

`style` - the given formatting style. For example, SHORT for "M/d/yy" in the US locale.

`aLocale` - the given locale.

Returns:

a date formatter.

getDateTimeInstance

```
public static final DateFormat getDateTimeInstance()
```

Gets the date/time formatter with the default formatting style for the default `FORMAT` locale.

This is equivalent to calling `getDateTimeInstance(DEFAULT, DEFAULT, Locale.getDefault(Locale.Category.FORMAT))`.

Returns:

a date/time formatter.

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getDateTimeInstance

```
public static final DateFormat getDateTimeInstance(int dateStyle,
                                                   int timeStyle)
```

Gets the date/time formatter with the given date and time formatting styles for the default `FORMAT` locale.

This is equivalent to calling `getDateTimeInstance(dateStyle, timeStyle, Locale.getDefault(Locale.Category.FORMAT))`.

Parameters:

`dateStyle` - the given date formatting style. For example, SHORT for "M/d/yy" in the US locale.

`timeStyle` - the given time formatting style. For example, SHORT for "h:mm a" in the US locale.

Returns:

a date/time formatter.

See Also:

`Locale.getDefault(java.util.Locale.Category)`, `Locale.Category.FORMAT`

getDateTimeInstance

```
public static final DateFormat getDateTimeInstance(int dateStyle,
                                                    int timeStyle,
                                                    Locale aLocale)
```

Gets the date/time formatter with the given formatting styles for the given locale.

Parameters:

`dateStyle` - the given date formatting style.

`timeStyle` - the given time formatting style.

`aLocale` - the given locale.

Returns:

a date/time formatter.

getInstance

```
public static final DateFormat getInstance()
```

Get a default date/time formatter that uses the SHORT style for both the date and the time.

Returns:

a date/time formatter

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `get*Instance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `DateFormatProvider` implementations. At a minimum, the returned array must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

Returns:

An array of locales for which localized `DateFormat` instances are available.

setCalendar

```
public void setCalendar(Calendar newCalendar)
```

Set the calendar to be used by this date format. Initially, the default calendar for the specified or default locale is used.

Any TimeZone and leniency values that have previously been set are overwritten by newCalendar's values.

Parameters:

newCalendar - the new Calendar to be used by the date format

getCalendar

```
public Calendar getCalendar()
```

Gets the calendar associated with this date/time formatter.

Returns:

the calendar associated with this date/time formatter.

setNumberFormat

```
public void setNumberFormat(NumberFormat newNumberFormat)
```

Allows you to set the number formatter.

Parameters:

newNumberFormat - the given new NumberFormat.

getNumberFormat

```
public NumberFormat getNumberFormat()
```

Gets the number formatter which this date/time formatter uses to format and parse a time.

Returns:

the number formatter which this date/time formatter uses.

setTimeZone

```
public void setTimeZone(TimeZone zone)
```

Sets the time zone for the calendar of this DateFormat object. This method is equivalent to the following call.

```
getCalendar().setTimeZone(zone);
```

The TimeZone set by this method is overwritten by a setCalendar call.

The TimeZone set by this method may be overwritten as a result of a call to the parse method.

Parameters:

zone - the given new time zone.

getTimeZone

`public TimeZone getTimeZone()`

Gets the time zone. This method is equivalent to the following call.

```
getCalendar().getTimeZone();
```

Returns:

the time zone associated with the calendar of DateFormat.

setLenient

`public void setLenient(boolean lenient)`

Specify whether or not date/time parsing is to be lenient. With lenient parsing, the parser may use heuristics to interpret inputs that do not precisely match this object's format. With strict parsing, inputs must match this object's format.

This method is equivalent to the following call.

```
getCalendar().setLenient(lenient);
```

This leniency value is overwritten by a call to `setCalendar()`.

Implementation Requirements:

A `SPACE_SEPARATOR` in the input text will match any other `SPACE_SEPARATOR`s in the pattern with lenient parsing; otherwise, it will not match.

Parameters:

`lenient` - when `true`, parsing is lenient

See Also:

`Calendar.setLenient(boolean)`

isLenient

`public boolean isLenient()`

Tell whether date/time parsing is to be lenient. This method is equivalent to the following call.

```
getCalendar().isLenient();
```

Returns:

`true` if the `calendar` is lenient; `false` otherwise.

See Also:

`Calendar.isLenient()`

hashCode

```
public int hashCode()
```

Returns the hash code for this `DateFormat`.

Overrides:

`hashCode` in class `Object`

Implementation Requirements:

This method calculates the hash code value using the value returned by `getNumberFormat()`.

Returns:

the hash code for this `DateFormat`

See Also:

`Object.hashCode()`

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this `DateFormat` for equality. Returns true if the object is also a `DateFormat` and the two formats would format any value the same.

Overrides:

`equals` in class `Object`

Implementation Requirements:

This method performs an equality check with a notion of class identity based on `getClass()`, rather than `instanceof`. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

`obj` - object to be compared for equality

Returns:

`true` if the specified object is equal to this `DateFormat`

See Also:

`Object.equals(Object)`

clone

```
public Object clone()
```

Overrides Cloneable

Overrides:

`clone` in class `Format`

Returns:

a clone of this instance.

See Also:

`Cloneable`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.